Reinforcement Learning: A Blackjack Agent

Konstantinos Bouros, Pavlos Zounits

Technical University of Crete

July 2, 2025



ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ
TECHNICAL UNIVERSITY OF CRETE

Environment creation

Our environment consists of a console where a human player or a trained agent can play and separate parts to conceive, train and test our agents, and visualize the produced policies.

Environment creation

Our environment consists of a console where a human player or a trained agent can play and separate parts to conceive, train and test our agents, and visualize the produced policies.

- The code was written manually in Google Colab

Environment creation

Our environment consists of a console where a human player or a trained agent can play and separate parts to conceive, train and test our agents, and visualize the produced policies.

- The code was written manually in Google Colab
- Inspired by GeeksForGeeks for Q-Learning (learning rate based on a paper) and console creation

Environment creation

Our environment consists of a console where a human player or a trained agent can play and separate parts to conceive, train and test our agents, and visualize the produced policies.

- The code was written manually in Google Colab
- Inspired by GeeksForGeeks for Q-Learning (learning rate based on a paper) and console creation
- Code for visualization from gymnasium

Environment creation

Our environment consists of a console where a human player or a trained agent can play and separate parts to conceive, train and test our agents, and visualize the produced policies.

- The code was written manually in Google Colab
- Inspired by GeeksForGeeks for Q-Learning (learning rate based on a paper) and console creation
- Code for visualization from gymnasium
- AI tools (mainly ChatGPT) used to debug and implement DQN

Phase 1 — Task 1

In the first task of phase 1 we considered an infinite card deck and performed the Q-Learning algorithm.

$$Q(s,a) \leftarrow Q(s,a) + \alpha(|Q(s,a)|)\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

Phase 1 — Task 1

In the first task of phase 1 we considered an infinite card deck and performed the Q-Learning algorithm.

$$Q(s,a) \leftarrow Q(s,a) + \alpha(|Q(s,a)|)\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

State:

- Player' s sum (11 to 20)
- Active aces (0 or 1)
- Dealer' s card (2 to 11)

Phase 1 — Task 1

In the first task of phase 1 we considered an infinite card deck and performed the Q-Learning algorithm.

$$Q(s,a) \leftarrow Q(s,a) + \alpha(|Q(s,a)|)\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

State:

- Player' s sum (11 to 20)
- Active aces (0 or 1)
- Dealer' s card (2 to 11)

Action:

- Hit or Stand

Phase 1 — Task 1

In the first task of phase 1 we considered an infinite card deck and performed the Q-Learning algorithm.

$$Q(s,a) \leftarrow Q(s,a) + \alpha(|Q(s,a)|)\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

State:

- Player' s sum (11 to 20)
- Active aces (0 or 1)
- Dealer' s card (2 to 11)

Action:

- Hit or Stand

- $\gamma = 1$
- $\epsilon = \max\{0.15, 0.999^i\}$
- $\omega = 0.8$
- $\alpha(|Q(s,a)|) = \frac{1}{|Q(s,a)|^\omega}$

*$|Q(s,a)|$: number of $Q(s,a)$ being chosen,
i: current episode

Phase 1 — Task 1 — Performance

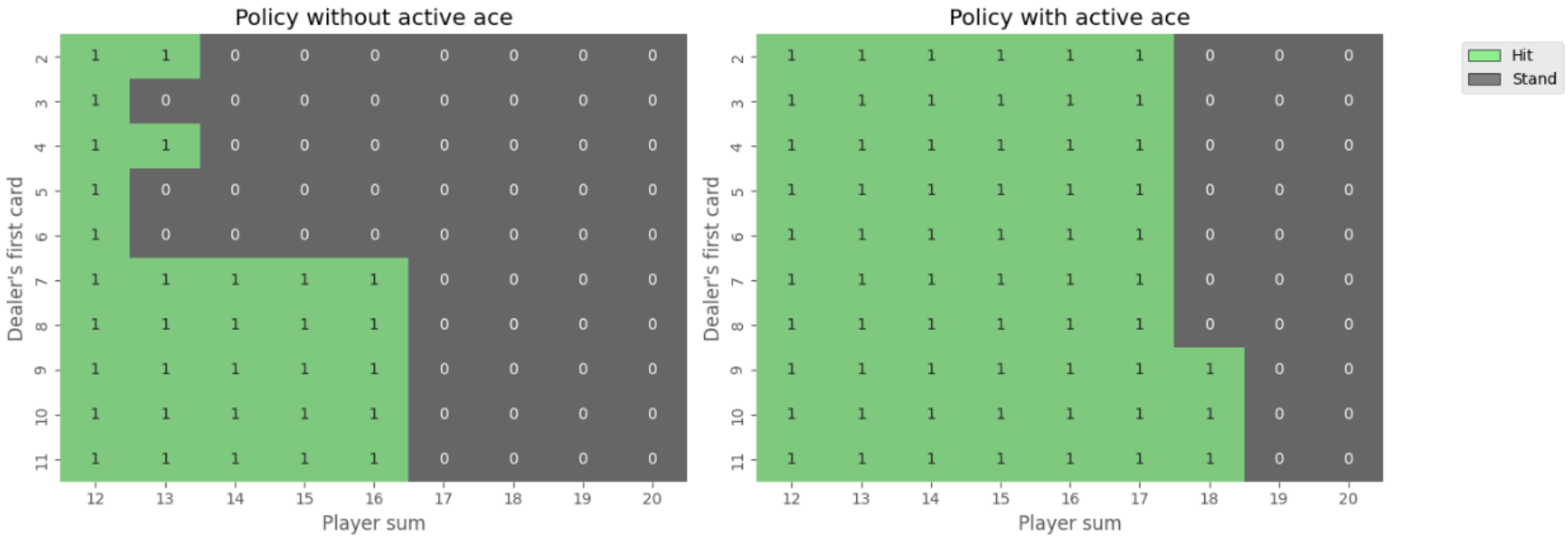


Figure: Learned policy with 10^6 episodes

Phase 1 — Task 2

Then, we started counting cards with the simple high/low method with a threshold of 2. From now we consider 1 card deck and refill it at the beginning of each game if the remaining cards are 10 or less. We trained the agent for 10^7 episodes.

Phase 1 — Task 2

Then, we started counting cards with the simple high/low method with a threshold of 2. From now we consider 1 card deck and refill it at the beginning of each game if the remaining cards are 10 or less. We trained the agent for 10^7 episodes.

```
Results for agent 2:
Wins: 44.64732 %, Losses: 47.43174 %, Ties: 7.92094 %

Results for agent 1 under task 2 environment:
Wins: 44.35662 %, Losses: 47.54525 %, Ties: 8.09813 %
```

Figure: Performances compared (with 1 deck, 10^7 testing episodes)

Phase 1 — Task 2

Then, we started counting cards with the simple high/low method with a threshold of 2. From now we consider 1 card deck and refill it at the beginning of each game if the remaining cards are 10 or less. We trained the agent for 10^7 episodes.

```
Results for agent 2:
Wins: 44.64732 %, Losses: 47.43174 %, Ties: 7.92094 %

Results for agent 1 under task 2 environment:
Wins: 44.35662 %, Losses: 47.54525 %, Ties: 8.09813 %
```

Figure: Performances compared (with 1 deck, 10^7 testing episodes)

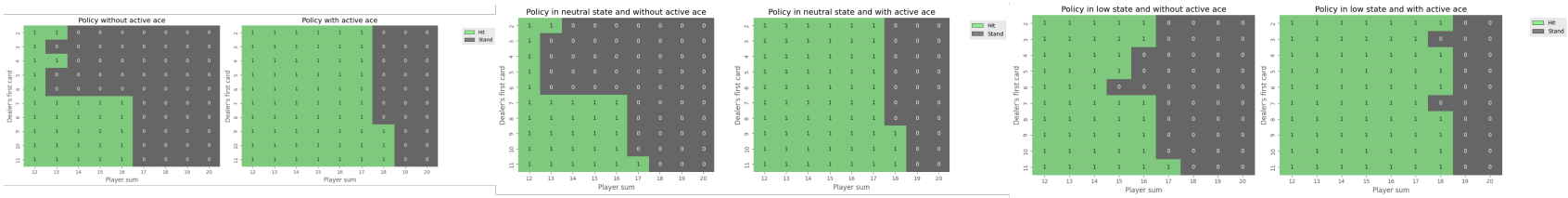


Figure: Learned policy (H, N, L)

Phase 2 — Task 1

For the second phase we made some modifications:

Phase 2 — Task 1

For the second phase we made some modifications:

- Added action Double (only on starting hand)

Phase 2 — Task 1

For the second phase we made some modifications:

- Added action Double (only on starting hand)
- Added action Split (only on starting hand with identical cards)

Phase 2 — Task 1

For the second phase we made some modifications:

- Added action Double (only on starting hand)
- Added action Split (only on starting hand with identical cards)
- Therefore we added `splittable` and `double_flag` as state variables

Phase 2 — Task 1

For the second phase we made some modifications:

- Added action Double (only on starting hand)
- Added action Split (only on starting hand with identical cards)
- Therefore we added `splittable` and `double_flag` as state variables
- Straight blackjack (with two cards) now pays $\times 1.5$

Phase 2 — Task 1

For the second phase we made some modifications:

- Added action Double (only on starting hand)
- Added action Split (only on starting hand with identical cards)
- Therefore we added `splittable` and `double_flag` as state variables
- Straight blackjack (with two cards) now pays $\times 1.5$
- Counting threshold: 3

Phase 2 — Task 1

For the second phase we made some modifications:

- Added action Double (only on starting hand)
- Added action Split (only on starting hand with identical cards)
- Therefore we added `splittable` and `double_flag` as state variables
- Straight blackjack (with two cards) now pays $\times 1.5$
- Counting threshold: 3
- Training and testing episodes: 10^7

Phase 2 — Task 1

For the second phase we made some modifications:

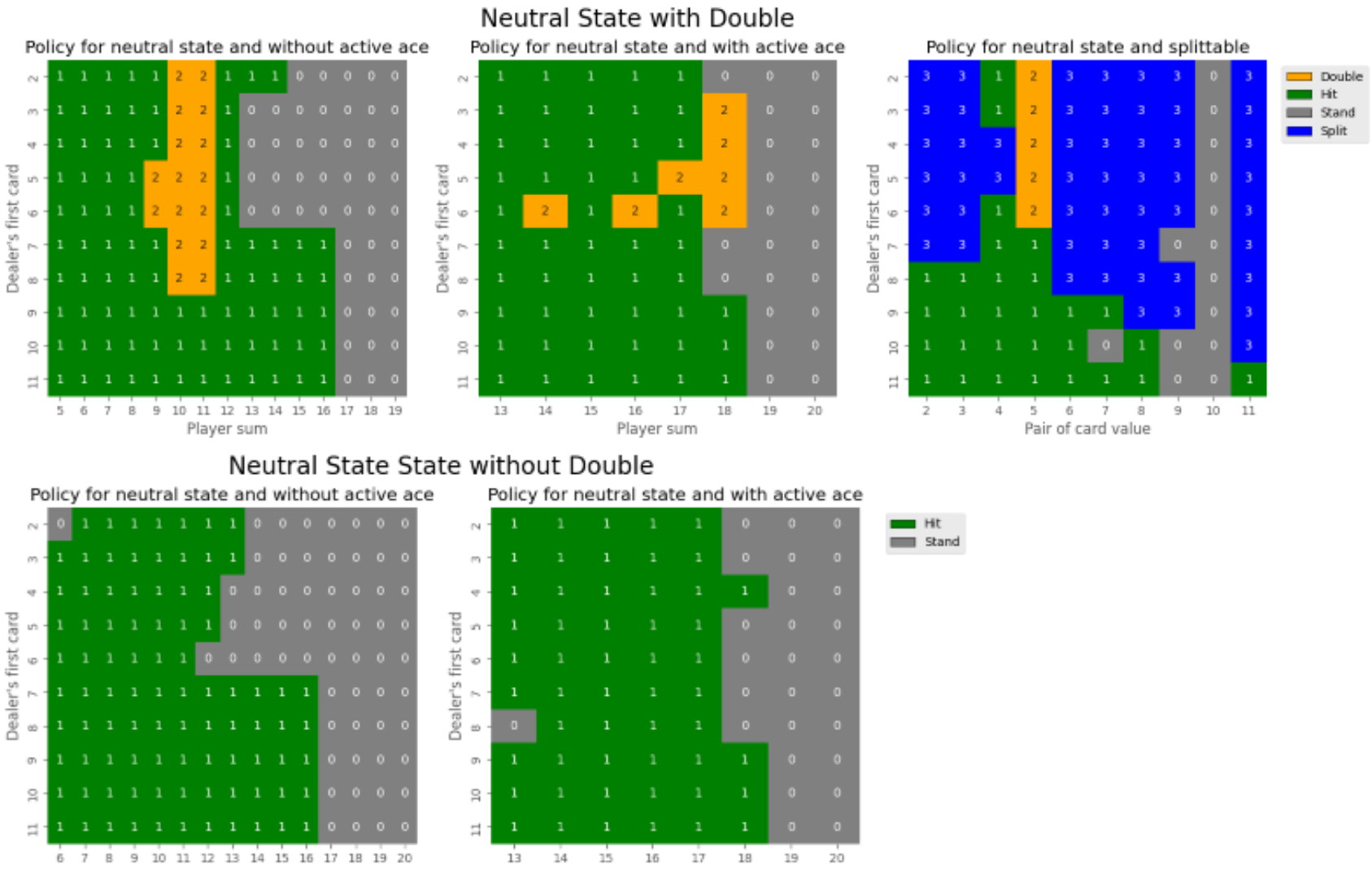
- Added action Double (only on starting hand)
- Added action Split (only on starting hand with identical cards)
- Therefore we added `splittable` and `double_flag` as state variables
- Straight blackjack (with two cards) now pays $\times 1.5$
- Counting threshold: 3
- Training and testing episodes: 10^7

```
Results for task 1:
Profit per episode: 0.01268085
```

Figure: Profit per episode

Note that profit per episode $= \frac{\text{rewards-losses}}{\text{episodes}}$

Phase 2 — Task 1 — Policy



Approach to Phase 2 — Task 2

- For the final task we first tried to implement DQN.

Approach to Phase 2 — Task 2

- For the final task we first tried to implement DQN.
- We switched to a more sophisticated model of counting. Instead of high/low, we used $[-7, -6, ..., 6, 7]$.

Approach to Phase 2 — Task 2

- For the final task we first tried to implement DQN.
- We switched to a more sophisticated model of counting. Instead of high/low, we used $[-7, -6, ..., 6, 7]$.
- We also included the bet in the state of the game policy.

Approach to Phase 2 — Task 2

- For the final task we first tried to implement DQN.
- We switched to a more sophisticated model of counting. Instead of high/low, we used $[-7, -6, ..., 6, 7]$.
- We also included the bet in the state of the game policy.
- 2 neural networks, 1 for betting and 1 for the game policy

```
class DQNetwork(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(DQNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.LayerNorm(256),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.LayerNorm(128),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.LayerNorm(64),
            nn.Linear(64, output_dim)
        )
```

Figure: Neural network(s)

Approach to Phase 2 — Task 2

- For the final task we first tried to implement DQN.
- We switched to a more sophisticated model of counting. Instead of high/low, we used $[-7, -6, ..., 6, 7]$.
- We also included the bet in the state of the game policy.
- 2 neural networks, 1 for betting and 1 for the game policy

```
class DQNetwork(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(DQNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.LayerNorm(256),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.LayerNorm(128),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.LayerNorm(64),
            nn.Linear(64, output_dim)
        )
```

Figure: Neural network(s)

- Training episodes: 10^5
- Testing episodes: $5 \cdot 10^5$
- Learning rate: 0.0002
- $\gamma = 1$
- $\epsilon = \max\{0.15, 0.9999^i\}$
- Batch size: 128
- Memory size: 10^4
- Target update freq: 700

Approach to Phase 2 — Task 2

We also used warm start training in this environment for the game policy by converting a tabular policy into supervised training data to improve the starting network.

Approach to Phase 2 — Task 2

We also used warm start training in this environment for the game policy by converting a tabular policy into supervised training data to improve the starting network.

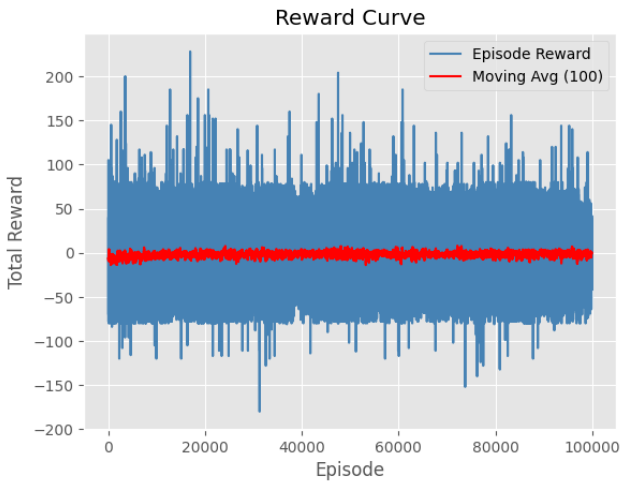


Figure: Rewards

Approach to Phase 2 — Task 2

We also used warm start training in this environment for the game policy by converting a tabular policy into supervised training data to improve the starting network.

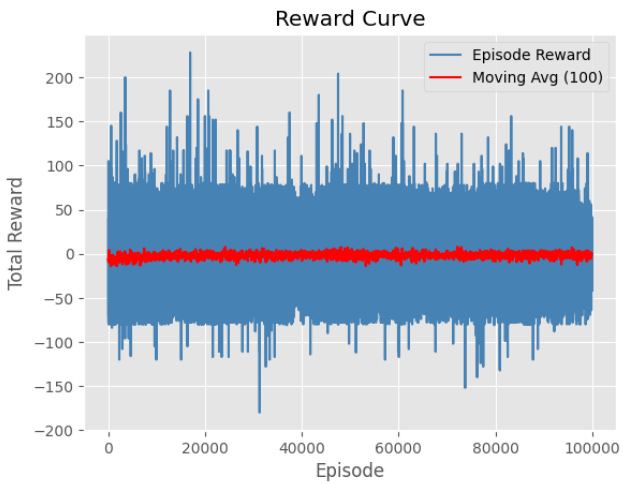


Figure: Rewards

```
Results for task 2 with DQN (true count):
Profit per episode: -0.391235
```

Figure: Profit per episode

But we could not make it work efficiently.

Approach to Phase 2 — Task 2

Then we tried to count the number of occurrences for each card (without warmup training).

Approach to Phase 2 — Task 2

Then we tried to count the number of occurrences for each card (without warmup training).

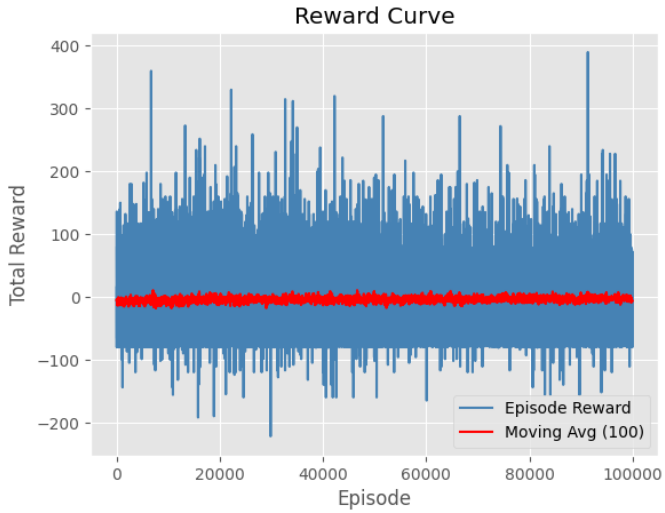


Figure: Rewards

Approach to Phase 2 — Task 2

Then we tried to count the number of occurrences for each card (without warmup training).

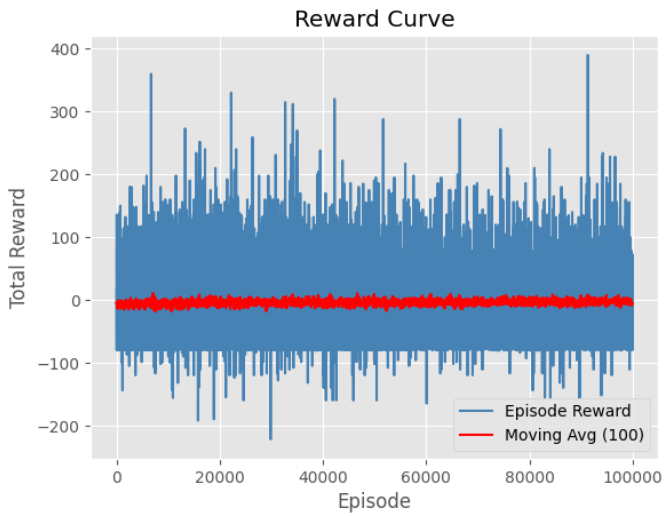


Figure: Rewards

```
Results for task 2 with DQN (complete count):
Profit per episode: -1.825087
```

Figure: Profit per episode

But we failed again! This is likely due to insufficient training or a simplistic network.

Phase 2 — Task 2

In the end, we decided to stick to Q-Learning with a true count method between $[-7, -6, ..., 6, 7]$. This gives a more accurate prediction of what card value to expect, compromising computational needs. We keep the same parameters and update ϵ to $\epsilon = \max\{0.15, 0.99995^i\}$.
We proceed as follows:

Phase 2 — Task 2

In the end, we decided to stick to Q-Learning with a true count method between $[-7, -6, ..., 6, 7]$. This gives a more accurate prediction of what card value to expect, compromising computational needs. We keep the same parameters and update ϵ to $\epsilon = \max\{0.15, 0.99995^i\}$.
We proceed as follows:

1. Train the agent with 1€ bets(like in the previous task)

Phase 2 — Task 2

In the end, we decided to stick to Q-Learning with a true count method between $[-7, -6, ..., 6, 7]$. This gives a more accurate prediction of what card value to expect, compromising computational needs. We keep the same parameters and update ϵ to $\epsilon = \max\{0.15, 0.99995^i\}$.
We proceed as follows:

1. Train the agent with 1€ bets(like in the previous task)
2. Then train the agent to bet the optimal amount in each counting state, given the previous game policy

Phase 2 — Task 2

In the end, we decided to stick to Q-Learning with a true count method between $[-7, -6, ..., 6, 7]$. This gives a more accurate prediction of what card value to expect, compromising computational needs. We keep the same parameters and update ϵ to $\epsilon = \max\{0.15, 0.99995^i\}$.
We proceed as follows:

1. Train the agent with 1€ bets(like in the previous task)
2. Then train the agent to bet the optimal amount in each counting state, given the previous game policy

```
Results for task 2 with tabular Q-learning:
Profit per episode: 0.4275433
```

Figure: Calculated profit per episode

Phase 2 — Task 2 — Performance

- For both steps, training and testing episodes: 10^7

Phase 2 — Task 2 — Performance

- For both steps, training and testing episodes: 10^7
- The produced game policy is similar to the one in the previous task.

Phase 2 — Task 2 — Performance

- For both steps, training and testing episodes: 10^7
- The produced game policy is similar to the one in the previous task.

```
---- Betting Policy: ----
Bet for true count -7: 1
Bet for true count -6: 1
Bet for true count -5: 1
Bet for true count -4: 1
Bet for true count -3: 1
Bet for true count -2: 1
Bet for true count -1: 1
Bet for true count 0: 20
Bet for true count 1: 28
Bet for true count 2: 30
Bet for true count 3: 40
Bet for true count 4: 38
Bet for true count 5: 34
Bet for true count 6: 35
Bet for true count 7: 40
```

Figure: Produced betting policy with a maximum bet of 40€

Phase 2 — Task 2 — 4 Decks alternative

We ran the same experiment for 4 decks instead of 1.

Phase 2 — Task 2 — 4 Decks alternative

We ran the same experiment for 4 decks instead of 1.

```
---- Betting Policy: ----
Bet for true count -7: 1
Bet for true count -6: 1
Bet for true count -5: 1
Bet for true count -4: 1
Bet for true count -3: 1
Bet for true count -2: 22
Bet for true count -1: 1
Bet for true count 0: 1
Bet for true count 1: 16
Bet for true count 2: 29
Bet for true count 3: 36
Bet for true count 4: 29
Bet for true count 5: 24
Bet for true count 6: 29
Bet for true count 7: 40
```

Figure: Produced betting policy with a maximum bet of 40€ and 4 decks

Phase 2 — Task 2 — 4 Decks alternative

We ran the same experiment for 4 decks instead of 1.

```
---- Betting Policy: ----
Bet for true count -7: 1
Bet for true count -6: 1
Bet for true count -5: 1
Bet for true count -4: 1
Bet for true count -3: 1
Bet for true count -2: 22
Bet for true count -1: 1
Bet for true count 0: 1
Bet for true count 1: 16
Bet for true count 2: 29
Bet for true count 3: 36
Bet for true count 4: 29
Bet for true count 5: 24
Bet for true count 6: 29
Bet for true count 7: 40
```

Figure: Produced betting policy with a maximum bet of 40€ and 4 decks

```
Results for task 2 with tabular Q-learning:
Profit per episode: 0.17013115
```

Figure: Profit per episode

The profit declines as we add more decks. This happens because we lose counting information, as the distribution of drawn cards tends to be uniform.

The End

Conclusion: we found a way to win against blackjack dealers with 0.42€ profit per game!

The End

Conclusion: we found a way to win against blackjack dealers with 0.42€ profit per game!



Thank you for your attention!